

04

THE DOUBLE-ENTRY LEDGER

My books balanced perfectly.
They explained nothing.

THREE CHARGES, THEN ONE CORRECTION

```
charge 1  201  balance = 4500000
charge 2  201  balance = 9000000
charge 3  201  balance = 13500000

-- then, by hand, outside the application:
UPDATE accounts SET balance = balance - 5000000
  WHERE merchant_id = '1111...' AND currency = 'NGN';
UPDATE 1
```

A refund adjustment, a support fix, a bad reconciliation. It happens on every payments team.

Six months later, someone asks why this balance is wrong.

THIS LOOKS FINE

transaction_id	account_id	direction	amount
-----+-----+-----+			
c73736ea...	506	CREDIT	4500000
c73736ea...	506	DEBIT	4500000
1f6c8626...	506	CREDIT	4500000
1f6c8626...	506	DEBIT	4500000
fc8b46cb...	506	CREDIT	4500000
fc8b46cb...	506	DEBIT	4500000
(6 rows)			

Every transaction has exactly one debit and one credit. Sum of debits equals sum of credits, to the naira. No orphan rows.

Every integrity check passes.

WHICH ENTRY EXPLAINS THE DIFFERENCE?

account_id	cached_balance	ledger_truth	discrepancy
-----+	-----+	-----+	-----

(0 rows)

Not a wrong answer. **No answer.**

The balance column says 8,500,000. The ledger nets to 0.
Nothing on record explains the gap, and nothing ever will.

WHY

Both legs were hitting the same account.

```
account 506:  +4500000  CREDIT
account 506:  -4500000  DEBIT
              _____
                   0
```

Debits equalled credits because I made that true by construction. Every pair cancels, so every account nets to zero, so no account has provably moved.

The check could not fail. That is not the same as passing.

THE FIX — MONEY MOVES BETWEEN ACCOUNTS

```
db.update("""
    INSERT INTO ledger_entries
        (transaction_id, account_id, direction, amount, currency)
    VALUES (?, ?, 'DEBIT', ?, ?)
    """, transactionId, settlementAccountId, amount, currency);

db.update("""
    INSERT INTO ledger_entries
        (transaction_id, account_id, direction, amount, currency)
    VALUES (?, ?, 'CREDIT', ?, ?)
    """, transactionId, merchantAccountId, amount, currency);
```

A charge debits **provider settlement** and credits **merchant available**. Two rows, two different accounts.

Entries are immutable — a trigger blocks UPDATE and DELETE. Corrections are reversing entries, never edits.

SAME SCRIPT. SAME MANUAL UPDATE.

```
transaction_id | account_id | direction | amount
-----+-----+-----+-----
578c5aee...   |          509 | CREDIT    | 4500000
578c5aee...   |          507 | DEBIT     | 4500000

account_id | cached_balance | ledger_truth | discrepancy
-----+-----+-----+-----
          509 |          8500000 |          13500000 | -5000000
(1 row)
```

One row. The account, the gap, and the exact amount nobody recorded.

The invariant that could not fail before now fails, correctly, and names the drifted account.

THE TRADE-OFF, MEASURED

READING ONE BALANCE	TIME	ROWS
cached column (PK lookup)	0.021 ms	1
summed from the ledger	139.5 ms	1,441,250

634 MB scanned, on 7.2 million entries. So the column stays — updated in the same transaction, explicitly a cache. The entries are the truth.

And yes, I tried an index on account_id. Postgres ignored it and scanned anyway: that account owns 20% of the table. 48 MB built in 1.4 seconds, never used once.

Post 05, Tuesday — transaction boundaries, and the provider call I left inside one.